Learning-Based Self-Healing Agents for Fault Detection and Recovery in Microservice Systems

Hadrian Ging Hin Chong
Faculty of Engineering and Computing
Dublin City University
Dublin, Ireland
hadrianginghin.chong2@mail.dcu.ie

Eamon Cullimore
Faculty of Engineering and Computing
Dublin City University
Dublin, Ireland
eamon.cullimore2@mail.dcu.

I. INTRODUCTION

In recent years, application development has transitioned from monolithic architecture to microservice-based systems, in which applications are broken down into smaller, independently deployable services [1]. This architecture design allows for greater scalability, flexibility, and quicker development cycles, and has thus emerged as the dominant paradigm for modern cloud-native systems [1]. However, these advantages come at the expense of increasing operational complexity, as errors can occur due to complex service interactions, distributed deployments, and dynamic runtime environments [2].

As the scale and complexity of microservices increases, troubleshooting and incident response become more time-consuming and error prone [3]. Manual monitoring and recovery mechanisms struggle to deal with the diversity and unpredictability of runtime errors, motivating research into automated operational support [4], [3]. In this context, Artificial Intelligence (AI) and machine learning techniques have emerged as promising enablers for decreasing human intervention by automatically detecting faults, diagnosing their causes, and executing recovery actions [5], [6].

This literature review explores AI-driven self-healing techniques for microservice-based systems, with a particular focus on learning-based runtime approaches. The review examines how machine learning helps decision-making for recovery cause analysis, and verification and validation techniques that ensure automated remediation remains safe and reliable [4], [5]. Across these themes, recurring methodological patterns are identified, which includes telemetry-driven learning, graph and trace-based modelling, and reinforcement learning control [6], [7], [5]. Finally, the review highlights open challenges such as robustness under concept drift, explainability, and ensuring correctness of autonomous self-healing behaviour.

II. The Shift from Monolithic to Microservice

Understanding the change in applications from Monolithic to Microservice Architecture is a key point in self-healing AI and our general topic. The literature below characterizes the change in application style not as a simple decision made, but as a series of complex trade-offs between the strengths and weaknesses of each style of development. The change is defined simply by the shift away from large application blocks to smaller, faster and scalable applications.

Within this, we can split our theme into smaller subthemes describing the complex trade-offs between both development paradigms which helps deepen our understanding of the broader topic in integrating AI self-healing systems into Microservices. The themes below focus on Microservices and their unique benefits and drawbacks and will help understand the performance benefits of Microservices against Monolith Architecture, the main drivers for the abandonment of Monolithic Architecture and finally the unique case of using Microservices with Edge Computing.

A. Performance Trade-Offs between Microservices and Monolithic Architectures

One interesting finding in the literature studied is that the evolution toward microservices does not inherently guarantee improved performance; in fact, for specific workloads, it often introduces a regression. Research by Blinowski et al. [8] challenges the assumption that distributed systems are faster, demonstrating through controlled experiments that monolithic architectures significantly outperform microservices in single-machine environments. Their data showed that for non-computationally intensive tasks, a monolith could handle over double the request volume of its microservice counterpart in a .NET environment.

This performance gap is largely attributed to the "communication overhead" apparent in the evolutionary shift. Liu et al. *[9]* and related studies emphasize that decoupling services replaces fast in-memory function calls with slower network requests. For lightweight services, the computational cost of serializing data and managing network traffic can equal the execution time of the business logic itself, effectively doubling the workload. Consequently, the literature suggests that the evolution to microservices is justified only when the goal is scaling throughput across multiple nodes, rather than optimizing the speed of individual requests.

B. *Drivers and Challenges of Changing Development Paradigms*

If performance can suffer, the literature isolates independent scalability as the primary driver for the change in development style. Christian et al. and Pathak et al. *[10], [11]* argue that organizations accept the complexity of microservices because monoliths hit a scalability wall where the entire application must be duplicated to scale a single feature. This inability to isolate high-demand components encourages the migration.

However, this evolution requires sophisticated migration strategies. Velepucha and Flores [1] highlight that successful evolution relies on specific patterns, most notably the Strangler Application Pattern, which incrementally replaces monolithic functionality. However, they also identify critical anti-patterns that can derail this evolution, such as the Shared Data anti-pattern. The literature warns that while keeping a shared database during the transition phase is common, maintaining it permanently negates the benefits of the evolution, creating a distributed Monolith system that suffers from the worst of both worlds having both the complexity of Microservices and the coupling of Monolith Architecture.

III. Learning-Based Decision-Making for Self-Healing Actions

Learning-based decision-making in self-healing systems aims to allow microservice architectures to autonomously select appropriate recovery actions when faults are detected. Unlike traditional approaches that rely on static rules or predefined thresholds, learning-based methods utilize runtime telemetry and historical system behaviour to adjust recovery strategies over time [4], [5]. This functionality is particularly crucial in microservice environments, where the diversity and unpredictability of failures make manual policy design increasingly difficult [12].

Throughout the reviewed literature, a similar decision-making pipeline emerges. Operational data, which includes metrics, logs, and traces, are continuously gathered and used to reflect the status of the system. The learning models are then trained to link these states with appropriate remediation actions, such as restarting services, rolling back incorrect deployments, modifying configuration, or reallocating resources [4], [5], [12].

A. *Autonomous Self-Healing Controllers*

Within this theme, a dominant line of research frames self-healing as a control problem, that is generally formalized as **Markov Decision Process (MDP)**. In this approach, system states encode runtime and configuration information, actions reflect potential recovery operations, and reward functions capture trade-offs between recovery speed, performance impact, and operational cost [5]. Reinforcement learning is then used to learn the best recovery policies by interaction with the running system [5], [12].

A representative instance of this approach would be QMConn, proposed by Samir et al., which uses Q-Learning to remediate configuration difficulties in Kubernetes-based microservice deployments [5]. QMConn frames recovery as a Markov Decision Process (MDP), in which runtime and configuration conditions define the system state and recovery actions such as service restarts, configuration updates, and rollbacks are selected to maximize long-term reward while considering the recovery costs into account [5]. In their evaluation, the authors demonstrated a mean time to recovery (MTTR) of about 13 minutes per error when a single configuration error is injected per service, and an average MTTR of 23.75 minutes per error under simultaneous multi-error conditions. Overall, the research presents how reinforcement learning can aid in adaptive, cost-conscious recovery decision-making for microservice misconfigurations in dynamic environments [5].

More recent research expands upon this controller-based paradigm by using large language models (LLMs) to improve fault interpretation within the decision-making loop. Yang et al., for example, present a hybrid approach in which an LLM analyses heterogeneous operational data, such as logs, metrics, and alarms, and converts it into structured fault representations [12]. These representations are then used by a deep reinforcement learning agent to choose relevant recovery steps. Although tested largely in cloud AI systems, the method is intended for containerized and distributed environments that roughly mimic recent microservice architectures. The findings suggest that integrating semantic fault interpretation and reinforcement learning enhances recovery

accuracy and efficiency when compared to standard rule-based approaches.

Taken together, these experiments show that autonomous self-healing controllers can learn from system behaviour over time, moving beyond predefined recovery rules. By integrating reinforcement learning with broader fault representations, such as those provided by LLMs, these techniques provide a flexible and adaptive foundation for automatic recovery in complex microservice systems.

Table 1 summarises representative learning-based decision-making approaches for self-healing, comparing learning methods, recovery actions, and reported improvements in recovery performance [5], [12].

Study	Learning Method	Target Problem	Recovery Actions	Key Results	Contribution
QMConn (Samir et al., 2024)	• Q-Learning • MDP-based control	• Configuration errors • Access-control faults in Kubernetes	• Service restart • Config update • Rollback	• MTTR ≈ 13 min/error (single fault) • MTTR ≈ 23.75 min/error • Faster than rule-based recovery	• Shows RL can adaptively select cost-aware recovery actions
Yang et al. (2025)	• LLM + Deep RL	• Fault interpretation • Recovery planning	• Dynamically chosen remediation steps	• Higher recovery accuracy than rule-based methods • Improved efficiency in complex faults	• Combines semantic fault understanding with learning-based control

Table 1: Learning-Based Decision-Making for Self-Healing

IV. Learning-Based Fault Understanding in Microservices

While Theme 2 focuses on choosing recovery actions, learning-based fault understanding addresses a more fundamental question: what went wrong and where. In microservice architectures, failures often propagate across several services, making it difficult to distinguish between symptoms & root causes [4], [13]. As a result, effective self-healing requires precise fault knowledge, particularly through anomaly detection & root cause analysis [14], [6]. The literature increasingly adopts learning-based techniques to model complicated service interactions and extract meaningful diagnostic insights from high-dimensional telemetry data [4], [14], [6].

A. Learning-Based Anomaly Detection

A significant body of research has consequently focused on learning models of typical system behaviour from runtime telemetry, allowing the detection of aberrations that signal defects or performance degradation and serve as triggers for downstream self-healing processes.

Recent studies have highlighted the importance of explicitly modelling service dependencies when detecting anomalies. For example, GAL-MAD presents an unsupervised anomaly detection system that jointly captures the structural relationships between microservices and the temporal dynamics of performance metrics [6]. By combining Graph Attention Networks (GATs) to model inter-service dependencies with Bi-LSTM layers to learn temporal patterns, GAL-MAD can detect anomalies that arise from complex interactions across service [6]. The use of SHAP-based explainability enables to model to identify which services and metrics contribute most to detected anomalies, allowing for more informed downstream diagnosis. The experimental results indicate good detection performance even under strongly imbalanced anomaly scenarios, with recall reaching up to 0.99, indicating the usefulness of graph-based learning for anomaly detection in microservices [6].

Other approaches highlight the advantages of multi-source telemetry integration for detecting anomalies in microservices systems. ServiceAnomaly shows that integrating distributed traces with numerous runtime profiling measures results in more accurate and informative anomaly detection than using a single data source [7]. The

approach simulates normal system behaviour with a Context Propagation Graph (CPG) that is annotated with both linear and non-linear correlations between six profiling metrics, allowing deviations to be discovered at both the execution-structure and performance-metric levels [7]. Experimental evaluation on the TeaStore and TrainTicket benchmark systems demonstrates that this integrated modelling obtains F1-scores of up to 85% to 86%, allowing for reasoning about where and how anomalies occur throughout service interactions [7].

In contrast to metric-and trace-based approaches, **DeepLog** focuses on anomaly detection using system logs [14]. By modelling log messages as sequential data with Long Short-Term Memory (LSTM) networks, DeepLog learns normal execution patterns without requiring labelled anomaly data [14]. Deviations from these learned patterns are flagged as anomalies in real time. Although DeepLog does not perform automated recovery, it enables online, per-log-entry anomaly detection and diagnosis from system logs. In evaluations on large datasets (e.g., HDFS and OpenStack), DeepLog delivers strong detection performance which has F-measure around 96% to 98%, while also supporting diagnosis via workflow reconstruction and incremental updates to reduce false positives over time [14].

Overall, these studies demonstrate that learning-based anomaly detection techniques, particularly those that incorporate structural dependencies, temporal behaviour, and various telemetry, are well adapted to detecting anomalous states in complex microservice systems [6], [7], [14]. However, simply detecting anomalies is insufficient for successful self-healing because it does not explain why failures occur or which service is to blame.

A comparative summary of learning-based anomaly detection techniques and their reported performance metrics is provided in *Table 2*.

Study	Data Sources	Model	Key Results	Strengths	Limitations
GAL-MAD (2025)	• CPU, memory, I/O • Network & response time	• GAT (service graph) • Bi-LSTM (temporal)	• Recall up to 0.99 • Strong performance under imbalance	• Captures service dependencies • SHAP explainability	• Less precise for fine-grained network faults
ServiceAnomaly (2024)	• Distributed traces • Profiling metrics	• Context Propagation Graph (CPG)	• F1 = 85% (TeaStore) • F1 = 86% (TrainTicket)	• Multi-source telemetry improves detection	• Depends on trace quality
DeepLog (2017)	• System logs	• LSTM sequence model	• F-measure = 96–98%	• Online, unsupervised detection • Fine-grained log analysis	• No direct recovery support

Table 2: Learning-Based Fault Understanding in Microservices

B. Learning-Based Root Cause Analysis in Microservice

To address the limitations of anomaly detection, a complementary line of research focuses on root cause analysis (RCA), which seeks to identify the underlying source of problems in a microservice architecture [4], [13]. Learning-based RCA techniques aim to reduce the manual work required to analyse huge dependency graphs and speed up fault localization [13]. These approaches utilize telemetry data such as traces, metrics, and resource indicators to determine a causal relationship between services and failures [12], [13]. *Table 3* presents a comparison of learning-based root cause analysis approaches, highlighting their diagnostic accuracy and suitability for self-healing microservice environments.

One notable example is TraceDiag, which addresses the difficulty of analysing large-scale distributed traces that often include hundreds of services, many of which are unrelated to given incident [15]. TraceDiag combines reinforcement learning and causal analysis to iteratively prune service dependency graphs, reducing redundant

components while retaining those most likely to be involved in the failure [15]. By learning interpretable pruning algorithms based on latency, anomaly indicators, and correlation patterns, TraceDiag significantly enhances both the efficiency & accuracy of root cause localisation [15]. The experimental results had shown that it consistently ranks true root cause higher than existing correlation and ranking based approaches, while presenting diagnostic graphs that are compact and easy to comprehend.

A more deployment-oriented approach is presented by MicroRCA, focusing on practical root cause localization without the need for application-level instrumentation. MicroRCA correlates service response-time anomalies with container and host-level resource indicators, constructing an attributed graph that includes both service interactions and co-location effects [3]. When anomalies are found, the technique extracts an anomalous subgraph and uses Personalized PageRank to rank likely faulty services. Experimental evaluation on a Kubernetes-based Sock-Shop benchmark shows that MicroRCA achieves 89% precision and 97% mean average precision (MAP) across 95 injected fault scenarios, including latency, CPU hog, and memory leak faults, outperforming state-of-the-art approaches such as MonitorRank and Microscope [3].

Together, these RCA techniques enhance learning-based anomaly detection by giving actionable diagnostic insights that allow for focused correction. Learning-based RCA is essential in enabling precise and efficient self-healing operations in complex microservice systems since it narrows down the set of likely defective services [15] [3].

Study	Data Sources	Technique	Results	Key Advantage	Self-Healing Value
TraceDiag	• Distributed traces	• RL-based graph pruning • Causal analysis	• Higher precision than BackTrace & GrootRank • Graphs reduced to <12 nodes	• Efficient RCA in large systems	• Enables faster, targeted remediation
MicroRCA	• Service response times • Container & host metrics	• Attributed graph • Personalized PageRank	• 89% precision • 97% MAP • 95 fault cases	• No instrumentation required • Models co-location effects	• Practical for production self-healing

Table 3: Root Cause Analysis

V. Verification and Validation of Self-Healing Systems

This theme addresses the critical need for systematic methodologies to ensure that self-healing mechanisms are both correct and effective. As self-healing systems transition from simple reactive scripts to complex, autonomous architectures including those powered by Large Language Models (LLMs) the risk of incorrect remediation or emergent, unanticipated behaviours increases. Research in this area is defined by two primary approaches: Formal Verification, which uses mathematical proofs to guarantee that a repair action solves a specific vulnerability without introducing new errors, and Chaos Engineering, which proactively injects faults at runtime to evaluate a system's resilience and its ability to withstand unexpected perturbations. While formal methods provide high-precision protection for code-level repairs, chaos engineering offers a broader, system-level validation of architectural resilience.

A. Formal Verification for Evaluation in Self-Healing

This sub-theme focuses on the integration of formal methods with modern AI techniques to provide mathematical certainty in the self-healing process.

In the paper authored by Tihanyi et al. [16], the researchers investigate how Large Language Models can be integrated with Bounded Model Checking (BMC) to automate software vulnerability repair while ensuring the fixes are correct. To address this,

the authors developed the ESBMC-AI framework, which utilizes the Efficient

SMT-based Context-Bounded Model Checker (ESBMC) to detect vulnerabilities and generate counterexamples. These counterexamples are then used to prompt an LLM to generate a code fix, which is subsequently re-verified by the BMC tool to ensure the violation no longer exists. Their findings demonstrate that integrating formal verification provides a necessary safety check for LLM-generated code, successfully automating the detection and repair of buffer overflows and arithmetic overflows with high accuracy.

Similarly, Ehrig et al. [17] investigate how self-healing behaviour can be formally verified to guarantee properties such as consistency, deadlock-freeness, and liveness. The authors propose a formal modelling and verification approach based on algebraic graph transformation, where failures and repair actions are explicitly represented as graph transformation rules. Their methodology is evaluated using a traffic light control system case study, demonstrating that static, tool-supported verification can identify correctness guarantees of self-healing mechanisms at design time. The results highlight the strength of formal methods in providing strong assurances but also reveal limitations in capturing runtime uncertainty and evolving system contexts.

Likewise, research by Carwehl et al. [18] explores how verification can be extended to runtime to cope with changing requirements in self-adaptive and self-healing systems. The study investigates whether runtime verification can continuously validate system behaviour as requirements evolve during execution. The authors propose an observer-based runtime verification approach that dynamically adapts verification properties using property specification patterns and property adaptation patterns. The methodology is evaluated using an embedded body sensor network prototype, showing that the approach can efficiently and correctly handle requirement changes without restarting the verification process. These findings suggest that advanced runtime verification complements design-time formal analysis by addressing the inherent dynamism of self-healing systems, thereby strengthening overall assurance.

B. Chaos Engineering for Resilience Evaluation

This sub-theme examines methodologies for subjecting self-healing systems to unanticipated conditions to test their adaptive capabilities at runtime. Naqvi et al. [19] asks whether chaos engineering can be used systematically to evaluate the correct behaviour of self-adaptive and self-healing systems. The authors proposed CHESS, a systematic approach that subject's systems to unexpected scenarios and perturbations and performed an exploratory study on a self-healing smart office environment to identify the approach's promises and limitations. Their results indicate that chaos engineering is a viable way to evaluate how well a managing system can withstand faults, filling a significant gap in traditional evaluation methods that often focus on design-time models rather than runtime behaviour.

Research conducted by Filho et al. [20] explores the evaluation and self-adaptation strategies used in microservice-based systems. The methodology involved a systematic literature mapping that analysed 21 primary studies to understand how self-adaptation, including self-healing, is currently implemented and validated. The findings revealed that while approximately 23.81% of studies address self-healing, the majority still rely on reactive strategies. This suggests a significant research opportunity for more advanced, proactive evaluation methods to handle the inherent complexity of distributed microservice architectures.

VI. Conclusion

This literature review had shown a significant shift towards AI-driven self-healing strategies in response to the increasing operational complexity of microservice-based systems [2], [4]. Collectively, these studies contribute to the field by demonstrating how learning-based techniques such as anomaly detection, root cause analysis, and reinforcement learning can reduce human intervention, improve fault diagnosis accuracy, and significantly reduce recovery times in distributed environments [14], [3], [5], [6].

A key strength of this body of work is its strong alignment with real-world systems. Many approaches are evaluated on realistic microservice platforms using production-like telemetry, including metrics, logs, and traces [3], [5], [7]. Advanced modelling techniques, particularly graph-based learning and reinforcement learning, are well suited to capturing service dependencies and dynamic system behaviour [5], [6], [15]. Recent efforts to improve explainability and validation further strengthen the practicality of autonomous self-healing systems [6], [12].

Nevertheless, significant weaknesses remain. Many studies are heavily relied on offline training and controlled fault injection, which limits their ability to manage changing workloads and unseen failure models [5], [15]. Furthermore, recovery decisions are frequently made without adequate

safety or correctness guarantees, introducing risk in production deployments [2], [12].

Future research should therefore concentrate on continuous learning, safety-aware recovery policies, and improved integration of learning-based decision-making and runtime verification [12], [15].

Overall, the reviewed literature provides a strong foundation for building runtime self-healing agents that autonomously detect errors, localize their root causes, and execute reliable recovery actions in dynamic microservice environments [6], [12].

VII. References

[1] V. Velepucha and P. Flores, "A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges," *IEEE Access*, vol. 11, pp. 88339–88358, 2023, doi: 10.1109/ACCESS.2023.3305687.

[2] G. Toffetti, S. Brunner, M. Blöchlinger, F. Dudouet, and A. Edmonds, "An architecture for self-managing microservices," in *Proceedings of the 1st International Workshop on Automated Incident Management in Cloud*, in AIMC '15. New York, NY, USA: Association for Computing Machinery, Apr. 2015, pp. 19–24. doi: 10.1145/2747470.2747474.

[3] L. Wu, J. Tordsson, E. Elmroth, and O. Kao, "MicroRCA: Root Cause Localization of Performance Issues in Microservices," in *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, Apr. 2020, pp. 1–9. doi: 10.1109/NOMS47738.2020.9110353.

[4] B. Magableh and M. Almi'ani, "A Self Healing Microservices Architecture: A Case Study in Docker Swarm Cluster," 2020, pp. 846–858. doi: 10.1007/978-3-030-15032-7_71.

[5] A. Samir, H. Dagenborg, and D. Johansen, "QMConn: A Self-Healing Controller for Microservices Using Q-Learning and Markov Decision Processes," in *2024 IEEE/ACM 17th International Conference on Utility and Cloud Computing (UCC)*, Sharjah, United Arab Emirates: IEEE, Dec. 2024, pp. 389–398. doi: 10.1109/UCC63386.2024.00060.

[6] L. Akmeemana, C. Attanayake, H. Faiz, and S. Wickramanayake, "GAL-MAD: Towards Explainable Anomaly Detection in Microservice Applications Using Graph Attention Networks," Apr. 26, 2025, *arXiv*: arXiv:2504.00058. doi: 10.48550/arXiv.2504.00058.

[7] M. Panahandeh, A. Hamou-Lhadj, M. Hamdaqa, and J. Miller, "ServiceAnomaly: An anomaly detection approach in microservices using distributed traces and profiling metrics," *J. Syst. Softw.*, vol. 209, p. 111917, Mar. 2024, doi: 10.1016/j.jss.2023.111917.

[8] G. Blinowski, A. Ojdowska, and A. Przybyłek, "Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation," *IEEE Access*, vol. 10, pp. 20357–20374, 2022, doi: 10.1109/ACCESS.2022.3152803.

[9] G. Liu, B. Huang, Z. Liang, M. Qin, H. Zhou, and Z. Li, "Microservices: architecture, container, and challenges," in *2020 IEEE 20th International Conference on Software Quality, Reliability and Security Companion (QRS-C)*, Dec. 2020, pp. 629–635. doi: 10.1109/QRS-C51114.2020.00107.

[10] J. Christian, Steven, A. Kurniawan, and M. S. Anggreainy, "Analyzing Microservices and Monolithic Systems: Key Factors in Architecture, Development, and Operations," in *2023 6th International Conference of Computer and Informatics Engineering (IC2IE)*, Sep. 2023, pp. 64–69. doi: 10.1109/IC2IE60547.2023.10331155.

[11] G. Pathak and M. Singh, "A Review of Cloud Microservices Architecture for Modern Applications," in *2023 World Conference on Communication & Computing (WCONF)*, Jul. 2023, pp. 1–7. doi: 10.1109/WCONF58270.2023.10235199.

[12] Z. Yang, Y. Jin, J. Liu, and X. Xu, "An Intelligent Fault Self-Healing Mechanism for Cloud AI Systems via Integration of Large Language Models and Deep Reinforcement Learning," Jun. 09, 2025, *arXiv*: arXiv:2506.07411. doi: 10.48550/arXiv.2506.07411.

[13] J. Bogatinovski, S. Nedelkoski, J. Cardoso, and O. Kao, "Self-Supervised Anomaly Detection from Distributed Traces," in *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, Dec. 2020, pp. 342–347. doi: 10.1109/UCC48980.2020.00054.

[14] M. Du, F. Li, G. Zheng, and V. Srikumar, "DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, in CCS '17. New York, NY, USA: Association for Computing Machinery, Oct. 2017, pp. 1285–1298. doi: 10.1145/3133956.3134015.

[15] R. Ding *et al.*, "TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems," Oct. 28, 2023, *arXiv*: arXiv:2310.18740. doi: 10.48550/arXiv.2310.18740.

[16] N. Tihanyi, Y. Charalambous, R. Jain, M. A. Ferrag, and L. C. Cordeiro, "A New Era in

Software Security: Towards Self-Healing Software via Large Language Models and Formal Verification," in *2025 IEEE/ACM International Conference on Automation of Software Test (AST)*, Apr. 2025, pp. 136–147. doi: 10.1109/AST66626.2025.00020.

[17] H. Ehrig, C. Ermel, O. Runge, A. Bucchiarone, and P. Pelliccione, "Formal Analysis and Verification of Self-Healing Systems," Mar. 2010, pp. 139–153. doi: 10.1007/978-3-642-12029-9_10.

[18] M. Carwehl, T. Vogel, G. N. Rodrigues, and L. Grunske, "Runtime Verification of Self-Adaptive Systems with Changing Requirements," in *2023 IEEE/ACM 18th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, May 2023, pp. 104–114. doi: 10.1109/SEAMS59076.2023.00024.

[19] M. A. Naqvi, S. Malik, M. Astekin, and L. Moonen, "On Evaluating Self-Adaptive and Self-Healing Systems using Chaos Engineering," in *2022 IEEE International Conference on Autonomic Computing and Self-Organizing Systems (ACSOS)*, Sep. 2022, pp. 1–10. doi: 10.1109/ACSOS55765.2022.00018.

[20] M. Filho, E. Pimentel, W. Pereira, P. H. M. Maia, and M. I. Cortés, "Self-Adaptive Microservice-based Systems - Landscape and Research Opportunities," in *2021 International Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*, May 2021, pp. 167–178. doi: 10.1109/SEAMS51251.2021.00030.